Exercices 33 à 36

Note: si vous rencontrez des difficultés sur l'une des questions de cette feuille d'exercices, vous pouvez lire les indications correspondantes à la fin de ce document. Il est cependant conseillé de ne le faire qu'en dernier recours: trop utiliser ces indications pourrait vous donner à tort l'impression que vous maîtrisez les exercices alors que ce n'est pas encore le cas.

Exercice 33 (écriture de pseudo-codes).

Pour chaque fonction ci-dessous, proposez un algorithme sous forme de pseudo-code.

(1) La fonction $\mathbf{f_1}$ prend en entrée deux entiers k et n tels que $0 \leqslant k \leqslant n$, et renvoie le nombre d'arrangements de k éléments parmi n, c'est-à-dire le nombre $n(n-1)\dots(n-k+1)$, avec la convention que ce produit vaut 1 si $k = 0$.

(2) La fonction $\mathbf{f_2}$ prend en entrée une liste de L de nombres réels et renvoie le plus petit élément strictement positif de L, ou le mot-clé `aucun` s'il n'y en a aucun.

(3) La fonction $\mathbf{f_3}$ prend en entrée une matrice M et renvoie le nombre de coefficients entiers de M.

(4) La fonction $\mathbf{f_4}$ prend en entrée une chaine de caractères s et renvoie deux chaînes de caractères, l'une constituée uniquement des voyelles (minuscules ou majuscules) de s, et l'autre constituée des autres caractères de s (à elles deux, ces deux chaînes ont donc le même nombre de caractères que s).

(5) La fonction $\mathbf{f_5}$ prend en entrée un polynôme P et renvoie $\lim\limits_{x\to+\infty} P(x)$, élément de $\mathbb{R} \cup \{-\infty, +\infty\}$.

Exercice 34 (traduction de pseudo-codes en Python).

Pour chaque fonction ci-dessous, définie par un pseudo-code, proposez une implémentation Python, puis prédisez le résultat sur deux exemples simples et vérifiez que votre code Python retourne les bons résultats.

(1)
```
fonction sans_cube(n)
| // entier → booléen
| // teste si n n'est divisible par aucun nombre de type p^3 avec p ⩾ 2 entier
| p ← 2
| tant que n ≠ 1
|     | si n est divisible par p
|     |     | si n est divisible par p^3
|     |     |     | retourner faux
|     |     | n ← n/p
|     |     | si n est divisible par p
|     |     |     | n ← n/p
|     | p ← p + 1
| retourner vrai
```

(2)
```
fonction somme_poly(P1,P2)
| // polynôme × polynôme → polynôme
| // calcule la somme de deux polynômes
| // chaque polynôme est représenté par une liste (la composante i est le coefficient de X^i)
| n1 ← taille de P1
| n2 ← taille de P2
| si n1 < n2
|     | retourner somme_poly(P2,P1) // on se ramène au cas n1 ⩾ n2
| P ← copie de P1
| pour k = 0, 1, ... n2 − 1
|     | P(k) ← P(k) + P2(k)
| retourner P
```

(3)

```
fonction texte(s,n)
| // chaîne de caractères × entier → liste (de chaînes de caractères)
| // extrait d'une chaîne de caractères s toutes les sous-chaînes maximales de longueur au moins n
| // composées uniquement de lettres (caractères 'a' à 'z', majuscules et minuscules)
| vérifier que n ⩾ 1
| ajouter le caractère '.' à la fin de s
| L ← liste vide // résultat
| m ← chaîne vide // mot extrait courant
| pour tout caractère c de s
|     | si c est une lettre
|     |     | ajouter c à m
|     | sinon
|     |     | si ( taille de m ) ⩾ n
|     |     |     | ajouter m à L
|     |     | m ← chaîne vide
| retourner L
```

(4)

```
fonction produit_matrices(A,B)
| // matrice × matrice → matrice
| // calcule le produit de deux matrices A et B
| // chaque matrice est représentée par une liste de lignes
| // chaque ligne est représentée par une liste de coefficients
| n ← nombre de lignes de A
| p ← nombre de colonnes de A
| vérifier que p = nombre de lignes de B
| q ← nombre de colonnes de B
| C ← liste vide
| pour i = 0, 1, ..., n − 1
|     | L ← liste vide
|     | pour j = 0, 1, ..., q − 1
|     |     | ajouter à L la valeur Σ_{k=0}^{p−1} A_ik B_kj
|     | ajouter L à C
| retourner C
```

Exercice 35 (type abstrait ensemble).

Prédisez le contenu de la variable X après exécution de chaque pseudo-code ci-dessous, puis traduisez le pseudo-code en Python (en utilisant le type `set`, ou bien `frozenset` lorsque c'est nécessaire) et vérifiez votre prédiction en l'exécutant.

(1)

```
| A ← ensemble_vide()
| ajouter_élément(0,A)
| B ← ensemble_vide()
| ajouter_élément(est_vide(A),B)
| ajouter_élément(appartient_à(0,A),B)
| X ← ensemble_vide()
| ajouter_élément(cardinal(B),X)
| X ← union(A,X)
```

(2)

```
| Y ← ensemble_vide()
| pour k = 2, 3, ..., 10
|     | ajouter_élément(k,Y)
| X ← ensemble_vide()
| tant que non est_vide(Y)
|     | x ← extraire_élément(Y)
|     | si x est pair
|     |     | ajouter_élément(2x + 1,X)
| ajouter_élément(cardinal(X),X)
```

(3)

$X \leftarrow$ ensemble_vide()
ajouter_élément(2,X)
ajouter_élément(3,X)
ajouter_élément(X,X)
ajouter_élément(X,X)
$T \leftarrow$ appartient_à(X,X)
ajouter_élément(T,X)

Exercice 36 (fougère de Barnsley).

On condidère une fonction **ifs** qui prend en entrée une famille $(A_k)_{k=0,1,2,\ldots}$ de matrices 2×2, une famille de vecteurs $(B_k)_{k=0,1,2,\ldots}$ de $\mathbb{R}^2$, ainsi qu'une liste de réels P et un entier n, et renvoie deux listes X et Y construites selon l'algorithme ci-dessous:

fonction ifs((A_k),(B_k),P,n)
- $x \leftarrow$ tirage aléatoire uniforme dans $[0,1]$
- $y \leftarrow$ tirage aléatoire uniforme dans $[0,1]$
- $X \leftarrow$ liste vide
- $Y \leftarrow$ liste vide
- répéter n fois
  - $t \leftarrow$ tirage aléatoire uniforme dans $[0,1]$
  - $k \leftarrow$ nombre d'éléments de P qui sont inférieurs à t
  - $\begin{pmatrix} x \\ y \end{pmatrix} \leftarrow A_k \begin{pmatrix} x \\ y \end{pmatrix} + B_k$
  - ajouter x à la fin de la liste X
  - ajouter y à la fin de la liste Y
- retourner X et Y

On décide de représenter la famille (A_k) par une liste de listes de listes `A`, de telle sorte que le coefficient (i,j) de A_k (en commençant la numérotation à 0) est donné par $A[k][i][j]$. De même, le coefficient i de B_k est donné par $B[k][i]$.

(1) Traduire en Python le pseudo-code de la fonction **ifs**. On rappelle qu'un tirage (pseudo-)aléatoire uniforme sur $[0,1]$ peut être obtenu par la fonction `random` du module `random`.

(2) Tester la fonction **ifs** à l'aide du code ci-dessous (voir le fichier `test_ifs.py`), en mettant la figure en plein écran à la fin pour bien voir les détails.

```
import matplotlib.pyplot as plt

A = [[[0,0],[0,0.16]],[[0.85,0.04],[-0.04,0.85]],
     [[0.2,-0.26],[0.23,0.22]],[[-0.15,0.28],[0.26,0.24]]]
B = [[0,0],[0,1.6],[0,1.6],[0,0.44]]
P = [0.01,0.77,0.88];

for n in range(1000):
    X,Y = ifs(A,B,P,1000)
    plt.plot(X[100:],Y[100:],'k.',markersize=0.1)

plt.axis('off')
plt.axes().set_aspect('equal')
plt.show()
```

Indications

33.1 S'inspirer du pseudo-code de la fonction factorielle vu au chapitre 3 p. 3

33.2 Faire une boucle sur les éléments de L et, dans la boucle, mettre à jour le plus petit élément strictement positif rencontré jusque là (initialisé à aucun).

33.3 Faire deux boucles imbriquées pour parcourir la matrice M, un boucle sur les indices de lignes et une boucle sur les indices de colonnes.
À l'intérieur de la double boucle, mettre à jour un compteur si le coefficient courant est entier.

33.4 Faire une boucle sur les caractères de s, et dans la boucle un test pour savoir si le caractère courant est une voyelle.

33.5 Considérer à part le cas où $P = 0$, puis le cas où P est de degré nul.
Dans les cas restants, considérer c, le coefficient dominant de P, et conclure en fonction de c.

34.1 La divisibilité de n par p se teste en considérant `n%p` (reste dans la division euclidienne de n par p).

34.2 Pour faire une copie de P_1 on peut utiliser la fonction `list()` ou la syntaxe `P1[:]`.

34.3 Attention, les chaînes de caractères sont immutables; pour ajouter le caractère '.' à la fin de s, il faut donc écrire `s = s+'.'`
Pour tester si un caractère `c` est une lettre minuscule, on peut écrire `c>='a' and c<='z'` ou même plus simplement `'a'<=c<='z'`

34.4 Le nombre de lignes de `A` est simplement `len(A)`; le nombre de colonnes de `A` est `len(A[0])`
Le coefficient A_{ik} s'écrit `A[i][k]`.
Pour calculer la somme cherchée, on peut utiliser une compréhension de liste

35.1 Les ensembles A et X contiennent des nombres, mais l'ensemble B contient des booléens.

35.2 Exécuter les boucles pas à pas et observer l'évolution des ensembles Y et X.

35.3 Attention à l'effet de ajouter_élément(X,X), qui ajoute à X un élément qui est lui-même un ensemble, ce qui n'est pas équivalent à $X \leftarrow X \cup X$.
Attention à l'effet de appartient_à(X,X), qui ne teste pas l'inclusion de X dans X mais l'appartenance de X (en tant qu'élement) à l'ensemble X.

36.1 Pour calculer k, on peut utiliser la somme (fonction `sum`) d'une expression génératrice simple.
Calculer x et y *en parallèle* en transcrivant en Python les formules de multiplication d'une matrice 2×2 (A_k) par une matrice 2×1 (B_k); les 4 coefficients de la matrice A_k sont `A[k][0][0]` et `A[k][0][1]` (première ligne), et `A[k][1][0]` et `A[k][1][1]` (deuxième ligne); les 2 coefficients du vecteur B_k sont `B[k][0]` et `B[k][1]`.

36.2 Compléter le fichier `test_ifs.py` avec la fonction `ifs` écrite à la question précédente.